

that can fit into a request. OpenAI's documentation also highlights that token counting helps estimate cost, route requests based on size, and avoid context-limit failures before an API call is made.

Token optimization is especially useful in repeated workflows such as customer support bots, retrieval-augmented generation systems, document summarizers, coding assistants, compliance reviewers, and agentic applications. In these scenarios, even a small reduction per request can produce meaningful savings when multiplied across thousands or millions of calls. More importantly, a shorter and cleaner prompt often reduces cognitive noise for the model, which can improve response consistency.

Prompt optimization techniques

Prompt optimization should be handled carefully because aggressive compression can damage meaning. Layered techniques should be used, starting with safe text cleanup and moving gradually towards semantic compression only when the user allows it. The following techniques are useful building blocks for a Python-based framework.

Remove filler phrases: Many prompts contain polite but unnecessary phrases such as “Could you please,” “I would like you to,” or “It would be great if.” These phrases are harmless in human communication, but they consume tokens without improving the model's understanding. For example, “Could you

please summarise the following article in five bullet points?” can be reduced to “Summarise the article in five bullet points.” The second version is shorter and equally clear.

Consolidate repeated instructions: A common source of wasted tokens is repeated intent. A prompt may say “be concise,” “keep the answer short,” and “avoid long explanations,” in three different places. A good optimizer should recognise that these instructions express the same constraint and retain only one clear version. The optimized instruction could simply be “Answer concisely.”

Deduplicate context: Long prompts often repeat the same background information across paragraphs. The optimizer can split the prompt into sentences or semantic chunks, compare their similarity, and remove near-duplicates. For example, “The customer is unhappy because the delivery was delayed” and “The delayed delivery made the customer dissatisfied” carry the same core meaning. The framework should keep the clearer sentence and remove the redundant one.

Preserve protected sections: Not every part of a prompt should be compressed. Legal clauses, safety rules, schema definitions, examples, API contracts, and compliance instructions may need to remain unchanged. The framework should allow users to mark protected sections so that optimization rules do not alter them. This approach is used by some open source prompt optimization tools, where protected tags help safeguard critical prompt fragments.

Convert verbose

instructions into compact structure: A paragraph of instructions can sometimes be replaced by a compact list. For instance, “You should classify the ticket, identify urgency, provide a short reason, and return the result in JSON format” can become: “Return JSON with: category, urgency, reason.” Structured prompting reduces ambiguity and usually consumes fewer tokens than long-form prose.

Use semantic

summarisation for long context: When a prompt includes lengthy background material, simple rule-based cleanup may not be enough. Semantic summarisation can condense the context into essential facts. This technique is useful for

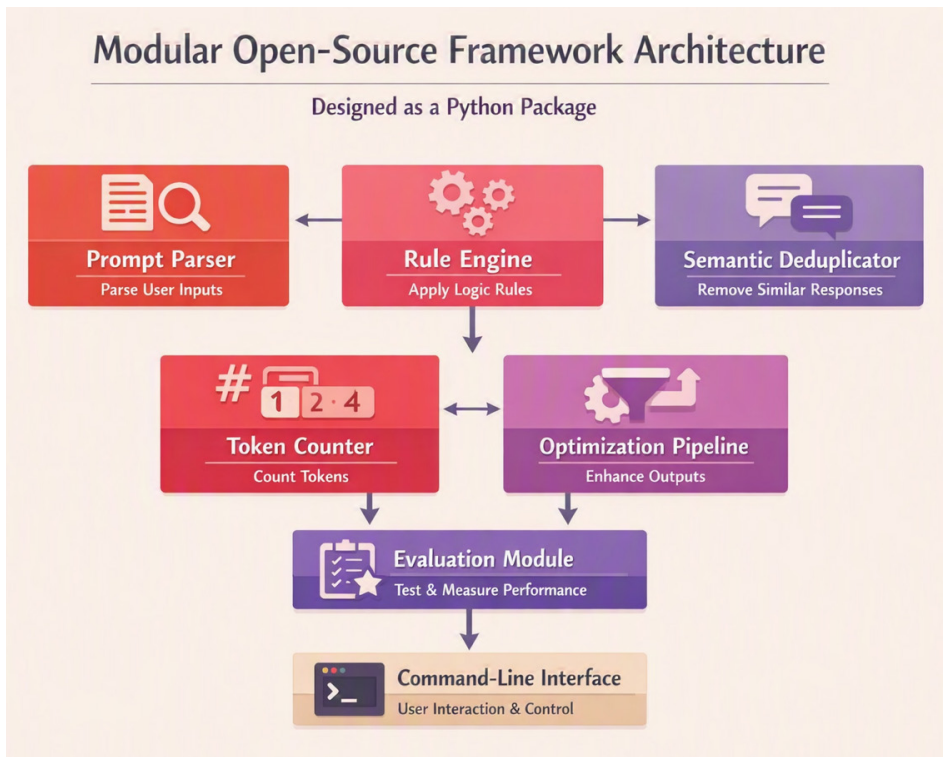


Figure 2: Reference architecture for token optimizer framework in Python